

HelixCluster

HelixCluster

Ein verteiltes Betriebssystem für AI-Computing - von Rechenzentrums-GPUs bis zu Edge-Handhelds, unter einer einzigen Steuerungsebene.

Zusammenfassung

Helix Cluster OS ist ein verteiltes Betriebssystem der nächsten Generation, das Rechenleistung über heterogene Knoten hinweg orchestriert - von Rechenzentrums-GPUs bis hin zu Edge-Single-Board-Computern (SBCs) und Handheld-Geräten. Es vereint HPC-Scheduling, Container-Orchestrierung, AI-/ML-Inferenz, föderierten Multi-Cluster-Betrieb und sichere Multi-Tenant-Sitzungen unter einer einzigen Steuerungsebene.

Kurzbeschreibung

Ein auf Go basierendes verteiltes Betriebssystem / ein GPU-Sharing-Compute-Cluster. Es integriert HPC-Scheduling (ein Omega-Modell mit zweistufigem Scheduler), Container-Orchestrierung, AI-Inferenz-Routing, Föderation und sichere Multi-Tenant-Sitzungen über heterogene Knoten hinweg, koordiniert durch SWIM-Gossip und Raft-Konsens mit quantensicherer Ende-zu-Ende-Verschlüsselung.

Ausführliche Beschreibung

Helix Cluster OS orchestriert Rechenlasten über radikal heterogene Hardware - von Rechenzentrums-GPUs über Edge-Single-Board-Computer bis hin zu Handheld-Geräten - unter einer einzigen Steuerungsebene. Dabei behandelt es einen Rack mit A100-GPUs und eine Handvoll SBCs als ein einheitliches, adressierbares Netzwerk statt als Dutzende inkompatibler Inseln. Es handelt sich um einen Go-Arbeitsbereich (ein Monorepo mit Git-Submodulen), der einen siebenstufigen Stack implementiert,

von der L0-Hardware-Ebene bis hin zu L7-Föderation und Observability, koordiniert durch vierzehn Microservices der Steuerungsebene. Die Knotenmitgliedschaft wird über SWIM-Gossip und Discovery verfolgt, sodass sich das Netzwerk selbst heilt, wenn Knoten hinzukommen oder ausscheiden. Stark konsistenter Zustand wird über Raft-Konsens verwaltet, organisiert in pro-Shard-Raft-Gruppen mit leseoptimierten Leaseholder-Lokalzugriffen und STONITH-Fencing, um sicherzustellen, dass ein partitionierter Knoten keinen gemeinsamen Zustand korrumpieren kann.

Die Workload-Platzierung erfolgt über einen zweistufigen Omega-Scheduler – mit optimistischer Nebenläufigkeit, ClassAd-Matching, Gang-Scheduling, wertbasierter Präemption und bedingungsgesteuerter Platzierung. Doch der Scheduler geht weiter als klassische HPC-Scheduler: Er berücksichtigt CO₂-Bilanz und Kosten/TCO, ermöglicht automatisches Skalieren in die Cloud (Burst-to-Cloud) und bietet Marktplatz-Adapter (Akash, io.net, RunPod, AWS Spot, Chutes), die es einem Job ermöglichen, auf gemietete Kapazitäten auszuweichen, wenn lokale Ressourcen erschöpft sind.

Endnutzer sehen von dieser komplexen Infrastruktur nichts direkt. Sie interagieren über ein klares Sitzungsmodell (Compute-Zuweisungen), ein interaktives WebSocket/PTY-Terminal, eine interne AI-Inferenz-Route und Lesezugriffe auf Pool-Auslastung. Sicherheit ist keine nachträgliche Ergänzung, sondern eine grundlegende Schicht: SPIFFE-Identitäten, Geräteattestierung (Challenge/Response, Proof-of-GPU-Work, Sealing), ein Exportkontroll-KYC-Gateway und ein quantensicherer Ende-zu-Ende-verschlüsselter Transport auf Basis eines hybriden Schlüsselaustauschs (X25519 + ML-KEM-768) mit AEAD-Datenschutz und Replay-Abwehr. Die Verschlüsselung ist so konzipiert, dass selbst heute abgefangener Datenverkehr auch gegen zukünftige Quantenangriffe geschützt bleibt.

Korrektheit wird nicht behauptet, sondern *nachgewiesen*: Deterministische Simulationstests (FoundationDB-ähnliche Seeded Runs, Fault Injection, Netzwerksimulation, Byte-für-Byte-Replay und ein Porcupine-Linearizability-Checker) reproduzieren verteilte Fehler auf Anforderung. Obligatorische gepaarte Mutationstests belegen, dass die Schutzmechanismen tatsächlich greifen. Architektur und Dokumentation werden durch mechanische Lints auf Konsistenz geprüft, die den Build sofort fehlschlagen lassen, sobald Realität und Dokumentation auseinanderklaffen.

Warum wir es entwickelt haben

Um AI- und HPC-Workloads über völlig unterschiedliche Hardware-Ebenen hinweg auszuführen – ohne separate Scheduler, Orchestratoren und Inferenz-Stacks zusammenflicken zu müssen. Und das mit der technischen Garantie, dass jede ausgelieferte Funktion *tatsächliches Endnutzerverhalten* nachweist (niemals grüne Tests mit Platzhaltern) und jede plattformspezifische Fähigkeit auf eine echte native Implementierung pro System zurückgreift (keine Linux-exklusiven Mocks). Das treibende Problem, zitiert in der Governance des Repositorys, ist der „Tests bestehen, aber die Funktion funktioniert nicht“-Fehlerzustand – ein Szenario, das das Projekt gezielt ausschließen soll.

Warum es ein Game-Changer ist

Es vereint fünf Komponenten, die normalerweise fünf separate Stacks erfordern – HPC-Scheduling, Container-Orchestrierung, AI-Inferenz, Multi-Cluster-Föderation und sichere Multi-Tenant-Sessions – in einer einzigen Steuerungsebene, die sich von Rechenzentrums-GPUs bis hin zu Edge-Handhelds erstreckt. Und das mit einer Präzision, die sonst nur spezialisierter Infrastruktur vorbehalten ist: Korrektheitsgarantien auf dem Niveau formaler Methoden (TLA+-Spezifikationen, deterministische Simulation, Linearizability-Prüfung) und quantensichere vertrauliche Datenübertragung – Sicherheitsstandards, die die meisten Orchestratoren erst gar nicht anstreben. Neben der technischen Überlegenheit sorgen kosten- und CO₂-bewusste Platzierung sowie Cloud-Marketplace-Burst dafür, dass es auch ein *wirtschaftlicher* Hebel ist: Der Scheduler sucht automatisch nach günstigerer, umweltfreundlicherer oder freier Kapazität, sodass dieselben Workloads weniger kosten und weniger Emissionen verursachen – ohne dass jemand den Job umschreiben muss.

Was innovativ ist

- **Deterministische Simulationsprüfung (DST)** – ein mit Seed-Werten arbeitender, vollständig reproduzierbarer Simulator, der Fehler, Taktabweichungen und Netzwerkpartitionen injiziert, diese Byte für Byte nachstellt und das Ergebnis durch einen Porcupine-Linearizability-Checker laufen lässt. So kann ein einmal gefundener Heisenbug auf Knopfdruck für immer reproduziert werden.
- **Omega-Modell-Zweitsatz-Scheduler** – optimistische Nebenläufigkeitsplatzierung mit ClassAd-Matching, Gang-Scheduling und wertbasierter Präemption, ein Shared-State-Design, das vielen Schemulern erlaubt, gleichzeitig auf ein

Cluster zuzugreifen, ohne einen zentralen Flaschenhals zu erzeugen.

- **Post-quantum E2EE / vertrauliche Inferenz-Infrastruktur** – ein hybrider Schlüsselaustausch aus X25519 und ML-KEM-768 mit pro Anfrage gebundenen Antwort-Schlüsselpaaren und AEAD mit Replay-Schutz (die kryptografischen Primitive sind real und getestet; der vollständige vertrauliche Multi-Node-Roundtrip ist explizit als PLANNED/gated gekennzeichnet).
- **Attestierungsbasiertes Vertrauen** – Knoten müssen *beweisen*, was sie sind: SPIFFE-Identität, Proof-of-GPU-Work, Gerätesiegelung, ein Exportkontroll-KYC-Gate und EU-AI-Act-konforme Dokumentengenerierung. Vertrauen wird hier durch Nachweise verdient, nicht durch Netzwerkposition vorausgesetzt.
- **Kosten- und CO₂-bewusste Orchestrierung** – TCO-Modellierung, CO₂-optimierte Platzierung, Cloud-Bursting, eine N+K-Failover-Reserve und Cloud-Marketplace-Adapter machen Preis und Emissionen zu primären Scheduling-Parametern – statt nachträglicher Überlegungen.
- **Multi-Raft-Konsens** – pro Shard Raft-Gruppen mit Leaseholder-lokalen Lesevorgängen für konsistente Low-Latency-Zugriffe, abgesichert durch STONITH-Fencing (IPMI / EC2 / Azure / SBD), sodass ein blockierter Knoten sofort und endgültig entfernt wird, statt den Zustand zu korrumpieren.
- **Anti-Drift-mechanische Lints** – archlint bricht den Build ab, sobald eine dokumentierte Komponente auf einen nicht existierenden Paketpfad verweist, und eine Docs-Chain-Engine hält Markdown / HTML / PDF / DOCX bytengenau konsistent. So kann die Dokumentation nicht stillschweigend von der Codebasis abweichen.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **„PASS-Bluff“ (Tests bestehen bei nicht-funktionalen Features).** Der Fehlermodus, den das gesamte Projekt ausmerzen soll: eine grüne Testsuite über Platzhaltern. Gelöst durch obligatorische paarweise Mutationstests – jedes Arbeitselement enthält einen benannten Wächtertest, der bei einer unabhängigen Code-Mutation *fehlschlagen* muss, bevor das Element als abgeschlossen markiert werden kann. So beweist ein bestandener Test, dass er echtes Verhalten prüft und nicht nur ein Mock.
- **Plattformübergreifende Parität (keine Linux-exklusiven Mocks).** Gelöst durch eine gemeinsame Schnittstelle, die per Build-Tags auf echte plattformspezifische Funktionen aufgeteilt wird – Linux cgroup / /proc / Kernel-WireGuard, macOS sysctl / vm_stat / IOKit / wireguard-go – und anschließend gegen ein unabhängiges OS-Orakel abgeglichen

wird. So meldet jede Plattform ihren nativen Zustand, statt einer Linux-Fiktion.

- **Verteilte Korrektheit unter Fehlern.** Gelöst durch deterministische Simulationstests und einen Linearizability-Checker, der Partitionierungen, Abstürze und Taktverschiebungen erzeugt und nachspielt, gestützt auf TLA+-Formalismen, die Konsens- und Scheduling-Invarianten festlegen, bevor auch nur eine Zeile Code geschrieben wird.
- **Dokumentations- und Architektur-Drift.** Gelöst durch archlint, das den Build bei jeder dokumentierten, aber nicht existierenden Paketzuordnung fehlschlagen lässt, sowie ein docs_chain-Verifizierungsgate ohne Ausweichmöglichkeit – Drift führt zum Build-Abbruch, nicht zu einer veralteten Wiki-Seite.
- **Ehrliche Abgrenzung unvollendeter Arbeit.** Der vertrauliche Multi-Node-Inferenz-Roundtrip ist bewusst hinter einem Ticket gesperrt und als „noch nicht end-to-end validiert“ gekennzeichnet – statt als ausgeliefert dargestellt zu werden. Dieselbe Disziplin gilt für das, was *noch nicht* erledigt ist, wie für das, was bereits funktioniert.

Technologie-Stack

- **Go (go.mod: 1.25 / Toolchain 1.26.4)** – die Kontrollplan-Sprache für einen ~30-Modul-Workspace; gewählt wegen der kostengünstigen Goroutine-Konkurrenz und statischen Binärdateien, die sich identisch vom Rechenzentrum bis zum Edge ausrollen lassen.
- **Zig (0.14+) + C/C++** – eingesetzt genau dort, wo Gos Laufzeitumgebung im Weg steht: bei Low-Level-Systemprimitiven und GPU-Kernen, die deterministische, speicherallokationsfreie Hardwarekontrolle benötigen.
- **gRPC + Protocol Buffers** – jede API-Schnittstelle zwischen Subsystemen (api/v1/) ist ein typisierter, versionierter Vertrag, sodass sich die vierzehn Microservices weiterentwickeln können, ohne sich gegenseitig zu brechen oder manuell Wire-Formate zu definieren.
- **Raft (etcd-raft) + SWIM-Gossip** – eine bewusste Trennung: Raft verwaltet den Zustand, der *stark konsistent* sein muss, während SWIM-Gossip für Mitgliedschaft und Discovery im großen Maßstab sorgt, wo Konsens zu aufwendig wäre.
- **PostgreSQL 16, Redis 7 Cluster, etcd v3.5, SQLite** – die richtige Datenbank für jede Aufgabe: PostgreSQL für dauerhafte relationale Daten, Redis für Hot-Cache, etcd für Koordination und eingebettetes SQLite für das knotenlokale HXC-Arbeits-element-Register.
- **NATS 2.10 (JetStream), Kafka 4.0 (KRaft), RabbitMQ 3.13** – drei Nachrichten-Backbones für drei Verkehrsmuster: NATS/JetStream für schnelle interne Events, Kafka für dauerhafte Hochdurchsatz-Streams, RabbitMQ für klassische Broker-Semantik.

- **WireGuard-Mesh + ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF** – WireGuard für ein schlankes Knoten-zu-Knoten-Netz, umhüllt von einem hybriden Post-Quanten-Handshake und AEAD-Records, sodass der Transport sowohl gegen klassische als auch Quantenangriffe abgesichert ist.
- **SPIFFE + JWT (HS256) + rollenbasierte RBAC + OPA** – gestaffelte Identitäts- und Berechtigungsverwaltung: SPIFFE für Workload-Identitäten, JWT für Tokens, rollenbasierte RBAC für grobe Zugriffskontrolle und OPA, um feingranulare Richtlinien als Code auszudrücken.
- **Prometheus v2.50, Grafana 10.4, Jaeger 1.55, W3C-Tracing** – Metriken, Dashboards und verteilte Traces mit W3C-Kontextfortpflanzung, sodass sich Anfragen über Dienste und Hardware-Ebenen hinweg verfolgen lassen.
- **HashiCorp Vault 1.16** – Geheimnisse und Schlüsselmaterial bleiben aus Code und Konfigurationen fern und werden unter Audit ausgegeben.
- **Docker Compose, Kubernetes (kustomize, gehärtete securityContext), Helm** – Compose für lokale Bereitstellung und Kubernetes/Helm mit gehärteten Security-Contexts für echte Deployments, wobei eine Definition über alle Umgebungen hinweg übernommen wird.
- **React + TypeScript + Vite (Node 20+)** – eine schnelle, typsichere Web-UI für Sitzungen, Terminals und Pool-Auslastung.
- **TLA+** – formale Spezifikation der Konsens- und Scheduling-Invarianten, sodass die am schwersten zu testenden Eigenschaften bereits auf Entwurfsebene bewiesen werden, bevor die Implementierung beginnt.

Status- & Ehrlichkeitshinweise

- **Status: in Entwicklung.** Die Version ist früh (0.1.0-dev). Mehrere fortgeschrittene Funktionen – vollständige vertrauliche Multi-Node-Inferenz-Roundtrips, Marktplatzabrechnung und attestierungsgesteuerte Planungspopulation – sind im Repository explizit als **GEPLANT / infrastrukturgesteuert** gekennzeichnet und werden **nicht** als voll funktionsfähig dargestellt. Die Abdeckungszahlen sind selbst gemeldet.
- **Lizenz: noch nicht festgelegt.** Nicht eindeutig deklariert; die URLs der Helm-Tabelle (HelixCluster/HelixCluster und helixcluster.io) sind nicht verifizierte Platzhalter, die nicht mit den tatsächlichen Remote-Quellen übereinstimmen.
- Die gebündelten LLM-Stack-Projekte (LLMOrchestrator, LLMProvider, LLMsVerifier) sind entkoppelte Submodule, keine innerhalb des Clusters gehosteten Modellserver.

Prioritätsstufe: Helix-primär (LLM-Infrastrukturcluster – das Rechensubstrat, das Inferenz- und Rechenlasten hosten kann).
Rangiert nach HelixTrack.